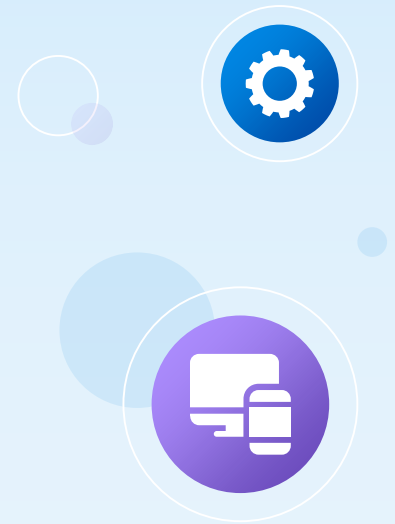
CSE212 Theory of Computation and Compilers

Parsers

Sara El-Metwally,
Associate Professor,
Computer Science Department

Table of contents



Parsing

- Parsing: Given a stream **s** of words and a grammar **G**, find a derivation in **G** that produces **s**.
- A CFG, G, is a set of rules, or productions, that describe how to form sentences.
- Sentence: a string of symbols that can be derived from the rules of a grammar
- The Context-free Grammers: For a language L, its **CFG** defines the set of strings of symbols that are valid in **L**.
- The collection of sentences that derive from G is called the language defined by **G**, denoted **L(G).**
- The set of languages defined by all possible CFGs is called the set of context-free languages.

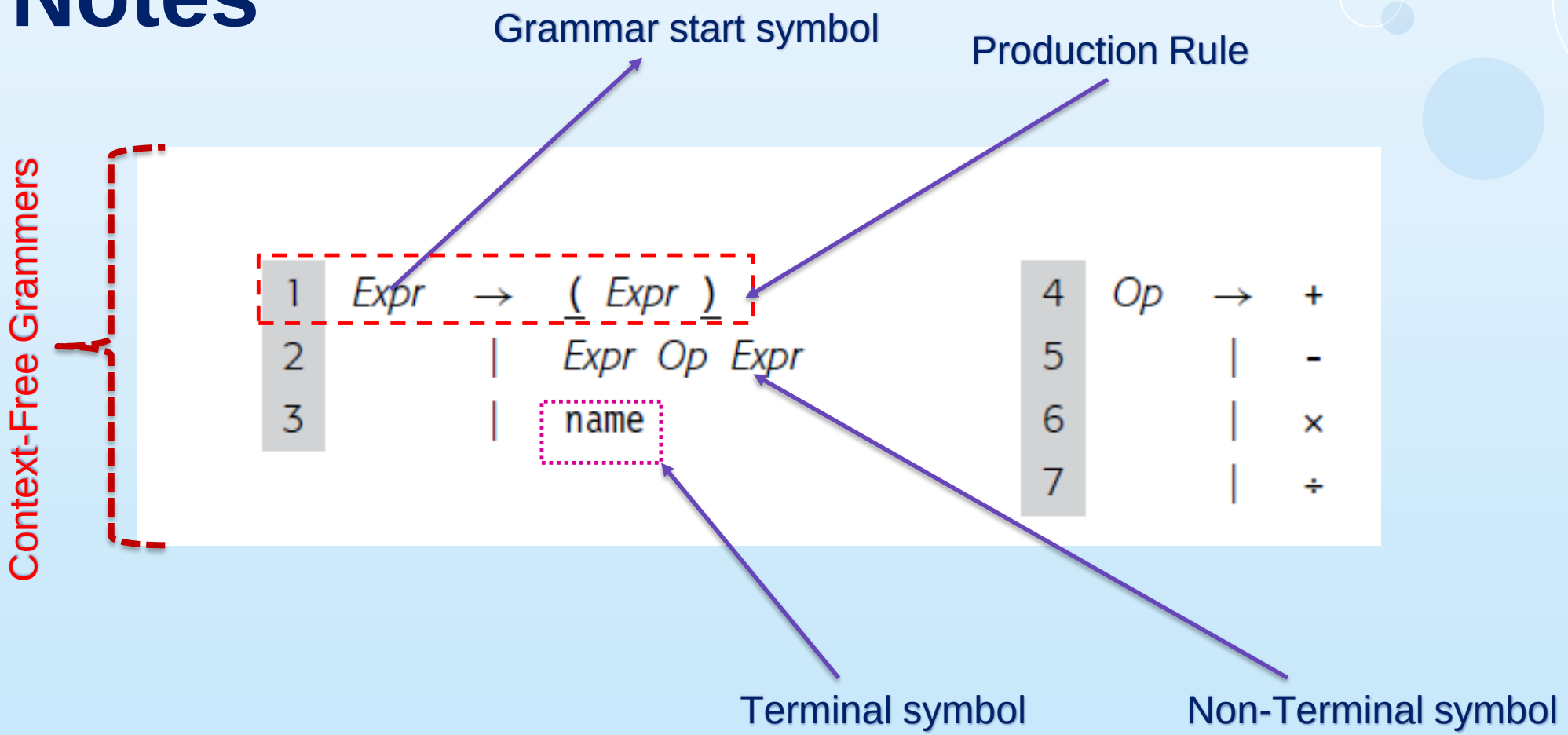
Context-Free Grammars

- Nonterminal symbol: a syntactic variable used in a grammar's productions.
- Terminal symbol: a word that can occur in a sentence.
- Derivation: a sequence of rewriting steps that begins with the grammar's start symbol and ends with a sentence in the language.

1	*Expr*	→	(*Expr*)
2		|	*Expr Op Expr*
3		|	name

4	*Op*	→	+
5		|	-
6		|	×
7		|	÷

Notes


Grammar start symbol
Production Rule
Context-Free Grammers
1 Expr → (Expr)
2 | Expr Op Expr
3 | name
4 Op → +
5 | -
6 | ×
7 | ÷
Terminal symbol
Non-Terminal symbol

Parsing

- Parse tree: a graph that represents a derivation; also called a syntax tree.

- Rightmost derivation: a derivation that rewrites, at each step, the rightmost non-terminal.

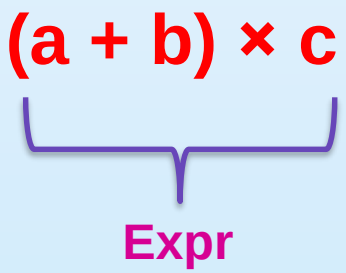
- Leftmost derivation: a derivation that rewrites, at each step, the leftmost nonterminal.

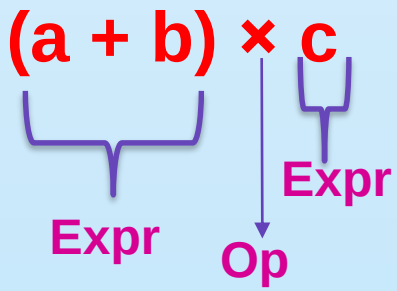
- Ambiguity: A grammar G is ambiguous if some sentence in L(G) has more than one rightmost (or leftmost) derivation.

- A grammar G is ambiguous if some sentence in L(G) has more than one parse tree.

Example: Right-most derivation of (a + b) × c

1	*Expr*	→	(*Expr*)	4	*Op*	→	+
2		|	*Expr Op Expr*	5		|	-
3		|	name	6		|	×
				7		|	÷





Example: Right-most derivation of (a + b) × c

1	*Expr*	→	(*Expr*)	4	*Op*	→	+
2		|	*Expr Op Expr*	5		|	-
3		|	name	6		|	×
				7		|	÷

Rule	Sentential Form
	Expr
2	Expr Op Expr

Left-most NT NT *Right-most NT*

Example: Right-most derivation of (a + b) × c

1	*Expr*	→	(*Expr*)	4	*Op*	→	+
2		|	*Expr Op Expr*	5		|	-
3		|	name	6		|	×
				7		|	÷

Rule	Sentential Form
	Expr
2	Expr Op Expr
3	Expr Op name
6	Expr × name
1	(Expr) × name
2	(Expr Op Expr) × name
3	(Expr Op name) × name
4	(Expr + name) × name
3	(name + name) × name

Example: Right-most derivation of (a + b) × c

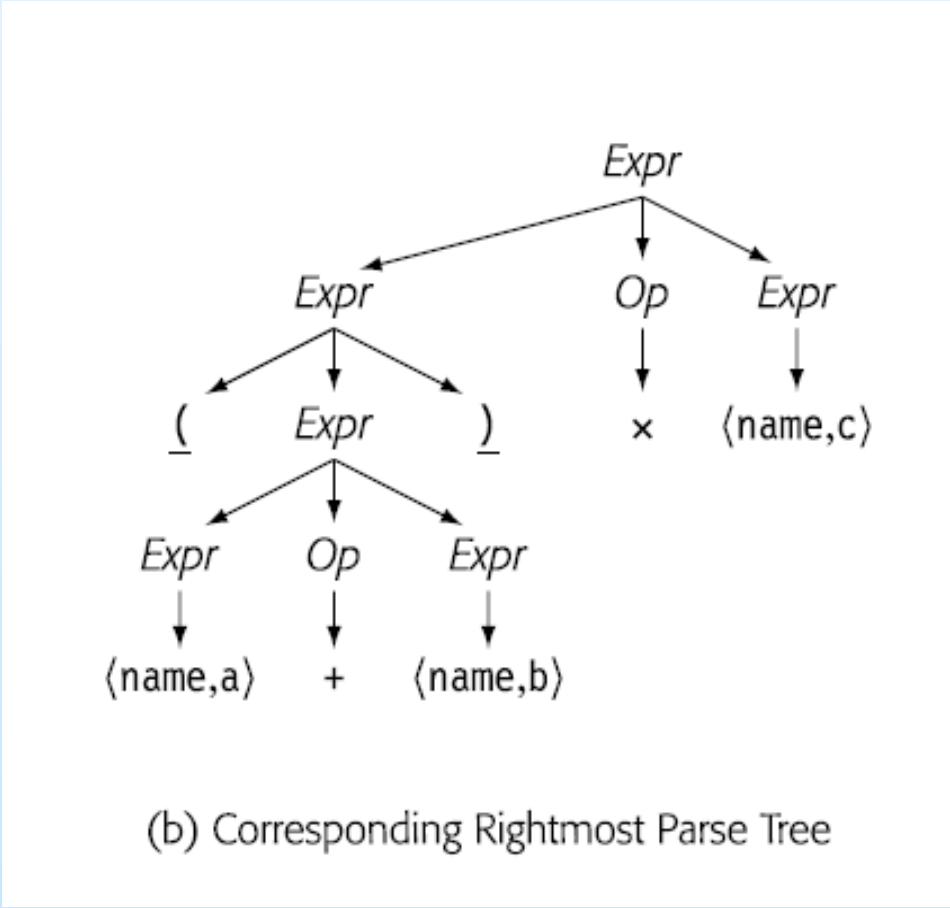
1	*Expr*	→	(*Expr*)	4	*Op*	→	+
2		|	*Expr Op Expr*	5		|	-
3		|	name	6		|	×
				7		|	÷

Grammers or CFG

Rule	Sentential Form
	Expr
2	Expr Op Expr
3	Expr Op name
6	Expr × name
1	(Expr) × name
2	(Expr Op Expr) × name
3	(Expr Op name) × name
4	(Expr + name) × name
3	(name + name) × name

Right-most Derivations

Example: Right-most derivation of (a + b) × c



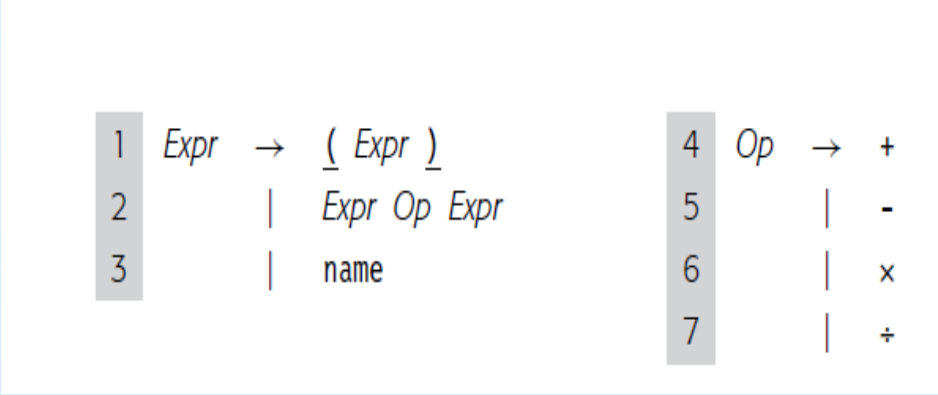
(b) Corresponding Rightmost Parse Tree

Right-most Parse Tree (Syntax Tree)

Rule	Sentential Form
	Expr
2	Expr Op Expr
3	Expr Op name
6	Expr × name
1	(Expr) × name
2	(Expr Op Expr) × name
3	(Expr Op name) × name
4	(Expr + name) × name
3	(name + name) × name

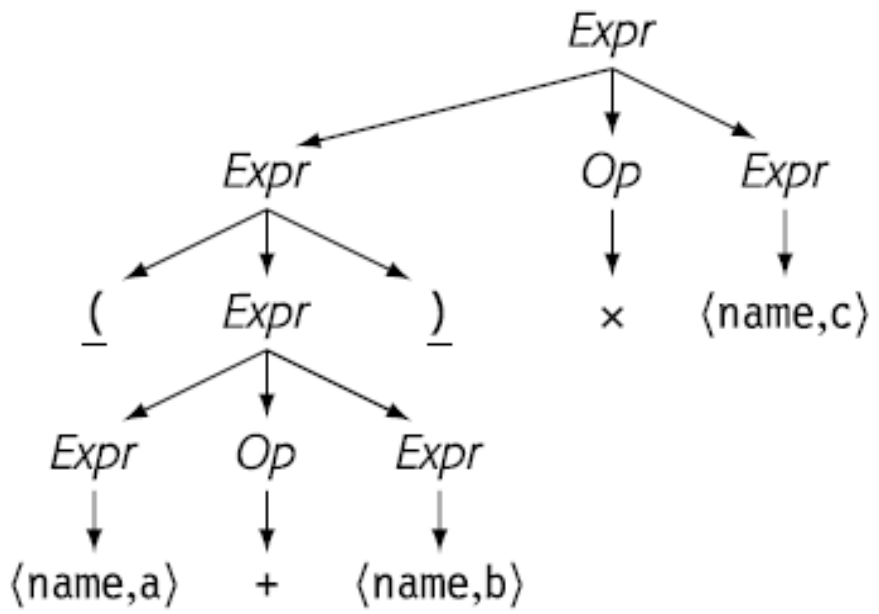
Right-most Derivations

Example: Left-most derivation of (a + b) × c



Rule	Sentential Form
	Expr
2	Expr Op Expr
1	(Expr) Op Expr
2	(Expr Op Expr) Op Expr
3	(name Op Expr) Op Expr
4	(name + Expr) Op Expr
3	(name + name) Op Expr
6	(name + name) × Expr
3	(name + name) × name

Example: Left-most derivation of (a + b) × c



(d) Corresponding Leftmost Parse Tree

Rule	Sentential Form
	Expr
2	Expr Op Expr
1	(Expr) Op Expr
2	(Expr Op Expr) Op Expr
3	(name Op Expr) Op Expr
4	(name + Expr) Op Expr
3	(name + name) Op Expr
6	(name + name) × Expr
3	(name + name) × name

Parsing

- Determine whether the following grammar is suitable for predictive parsing,

$$S \rightarrow (S)|SS|\varepsilon$$

Parsing

- Determine whether the following grammar is suitable for predictive parsing,

$$S \rightarrow (S)|SS|\varepsilon$$

Solution:

- Is this grammar is ambiguous? If yes: it is not suitable
- To prove the grammar ambiguity, Choose a sentence and Do a parse tree derivation

Parsing

- Determine whether the following grammar is suitable for predictive parsing,

$$S \rightarrow (S)|SS|\varepsilon$$

Sentence: ()()()

Parsing

- Determine whether the following grammar is suitable for predictive parsing,

$$S \rightarrow (S)|SS|\varepsilon$$

Sentence: ()()()

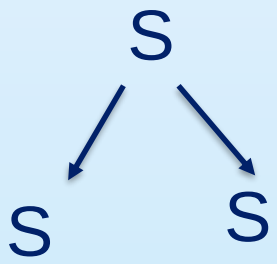
S

Parsing

- Determine whether the following grammar is suitable for predictive parsing,

$$S \rightarrow (S)|SS|\varepsilon$$

Sentence: ()()()

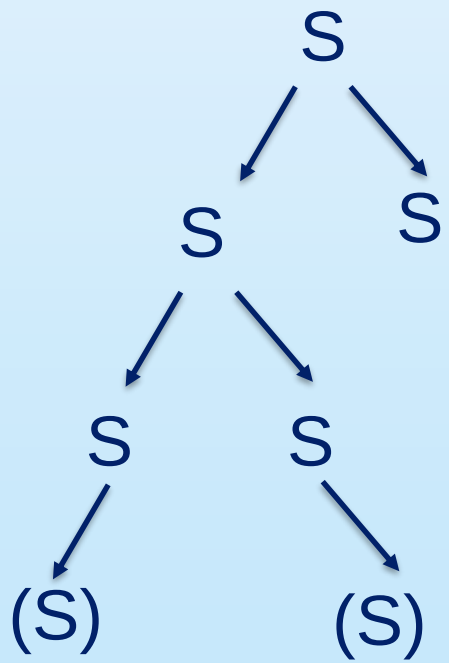


Parsing

- Determine whether the following grammar is suitable for predictive parsing,

$$S \rightarrow (S)|SS|\varepsilon$$

Sentence: ()()()

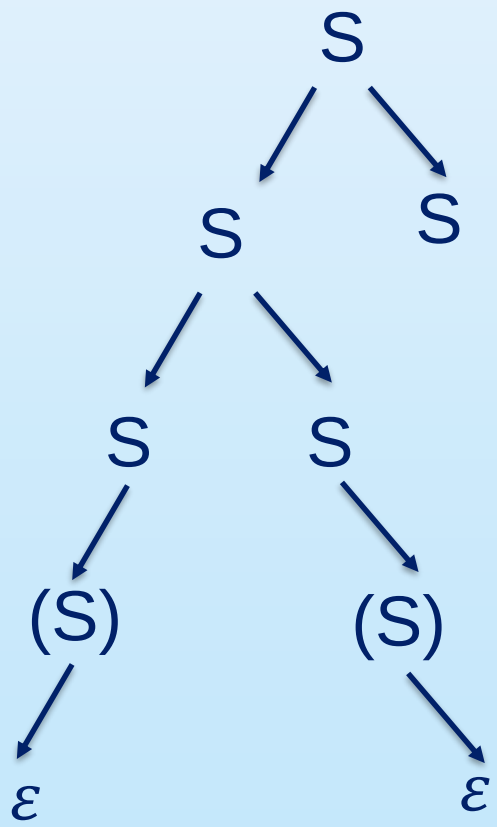


Parsing

- Determine whether the following grammar is suitable for predictive parsing,

$S \rightarrow (S)|SS|\varepsilon$

Sentence: ()()()

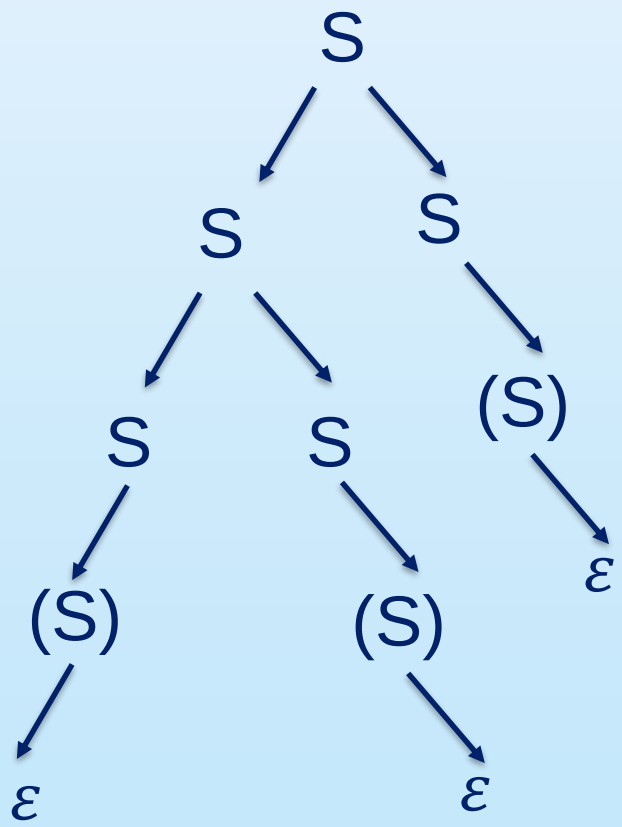


Parsing

- Determine whether the following grammar is suitable for predictive parsing,

$S \rightarrow (S)|SS|\varepsilon$

Sentence: ()()()

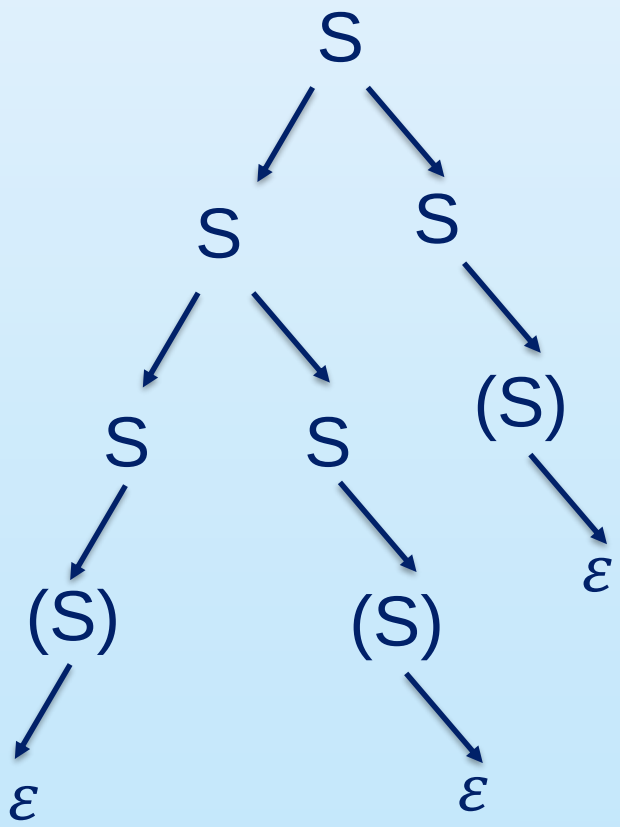


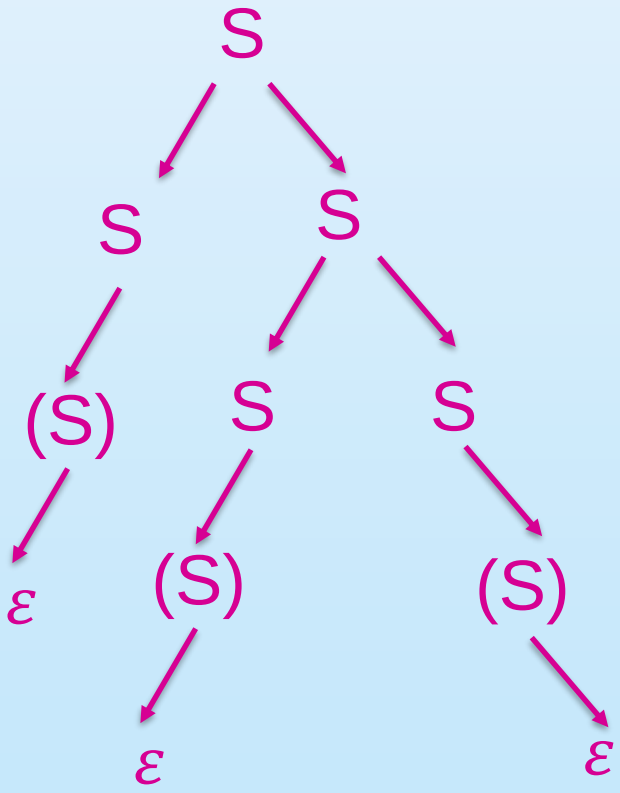
Parsing

- Determine whether the following grammar is suitable for predictive parsing,

$S \rightarrow (S)|SS|\varepsilon$

Sentence: ()()()





Parsing

- Consider the following two Grammars,
  - G1: S → $SbS|a$
  - G2: S → $aB|a$b, A → $AB|a$, B → $ABb|b$

- Q1: for G2, Define N, T, P, S

- Q2: Which of the following option is correct:
  - Only G1 is ambiguous
  - Only G2 is ambiguous
  - Both G1 and G2 are ambiguous
  - Both G1 and G2 are unambiguous

Parsing

- Consider the following two Grammars,
  - G1: S → $SbS|a$
  - G2: S → $aB|a$b, A → $AB|a$, B → $ABb|b$

- Q1: for G2, Define N, T, P, S
  - **Non-terminals N:** S, A, B
  - **Terminals T:** a, b
  - **Start Symbols S:** S
  - **Production Rules P:** S → $aB|a$b, A → $AB|a$, B → $ABb|b$

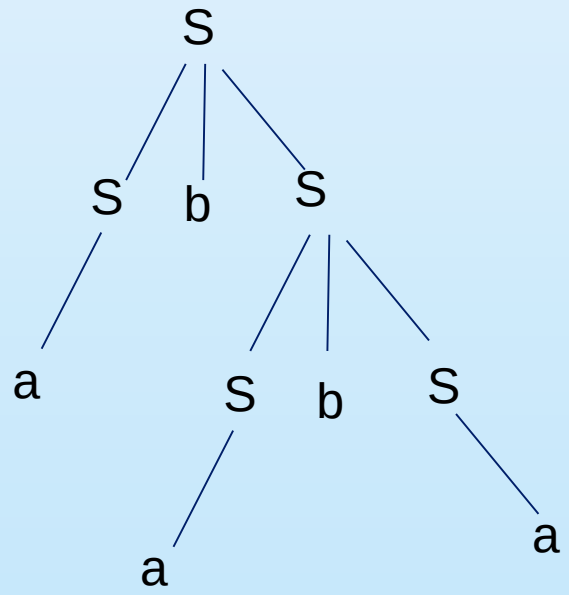
Parsing

- Consider the following two Grammars,
  - G1: S → $SbS|a$
  - G2: S → $aB|a$b, A → $AB|a$, B → $ABb|b$
- Q2: Which of the following option is correct:
  - Only G1 is ambiguous
  - Only G2 is ambiguous
  - Both G1 and G2 are ambiguous
  - Both G1 and G2 are unambiguous

Parsing

- Consider the following two Grammars,
  - G1: S → $SbS|a$

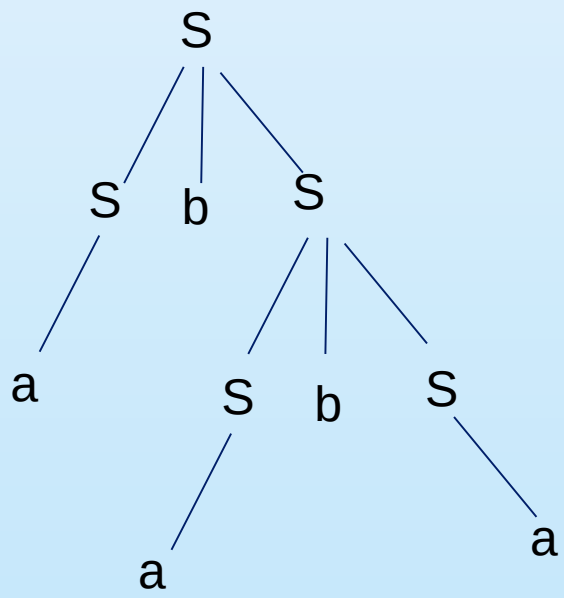
Consider a sentence: ababa

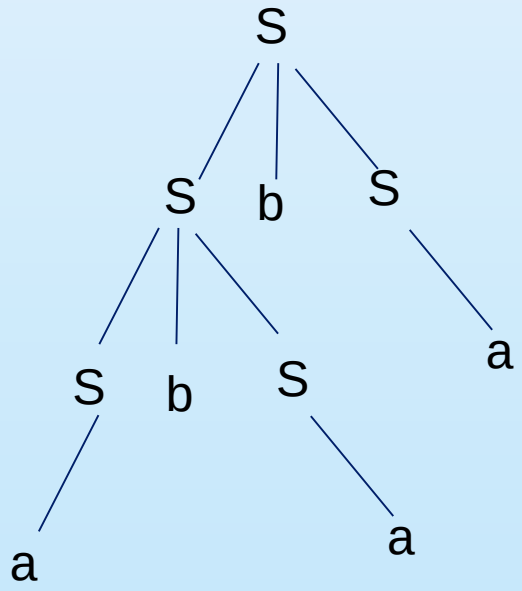


Parsing

- Consider the following two Grammars,
  - G1: S → $SbS|a$

Consider a sentence: ababa

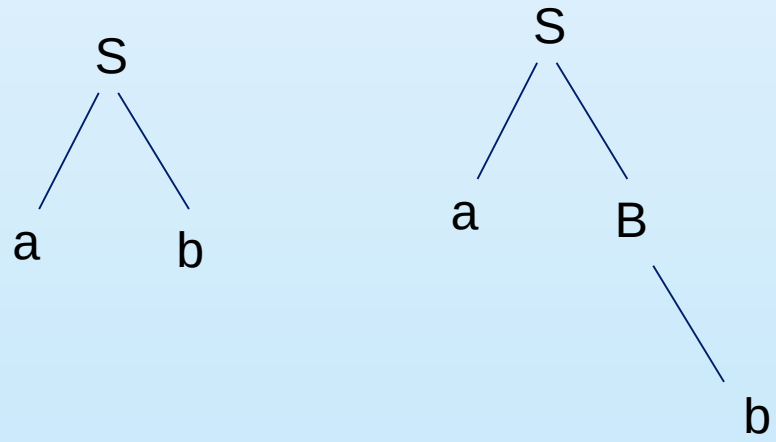




Parsing

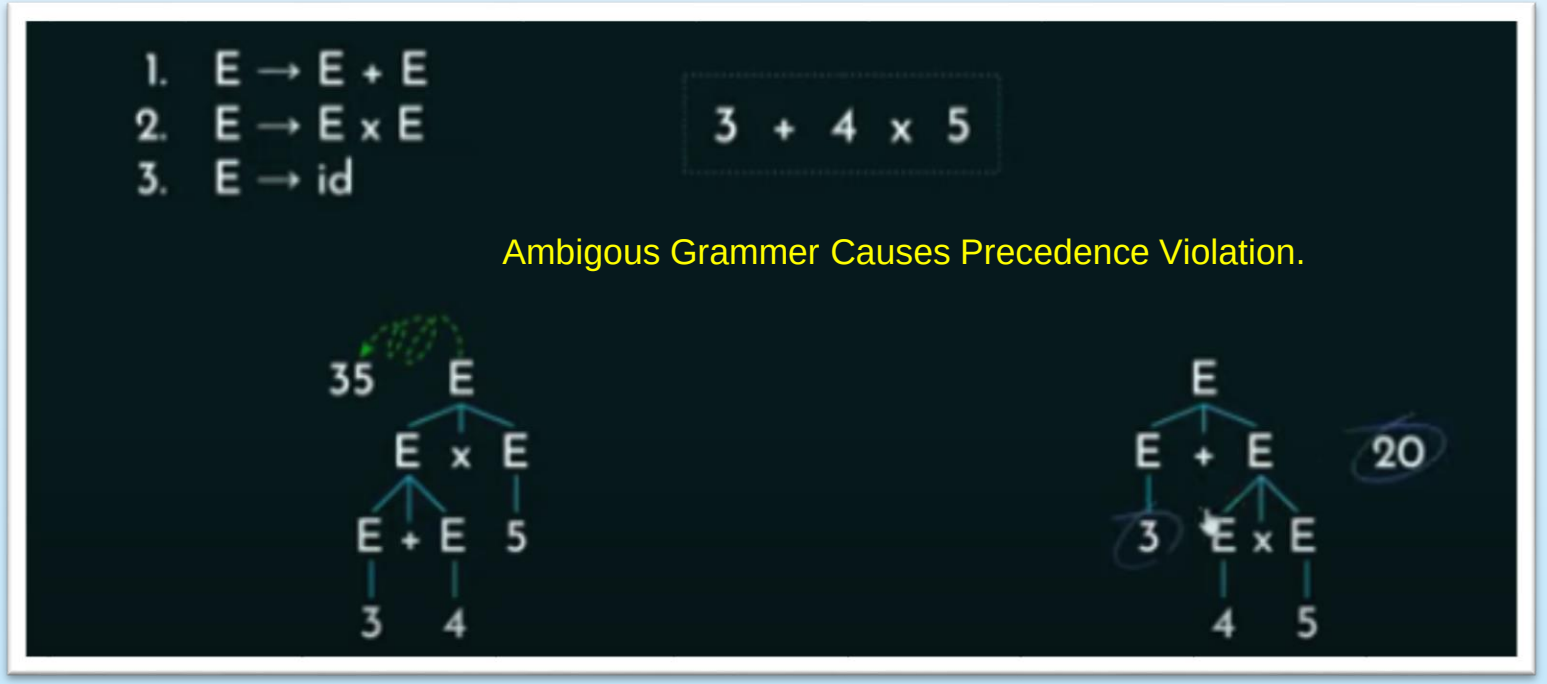
- Consider the following two Grammars,
  - G2: S → $aB|a$b, A → $AB|a$, B → $ABb|b$

Consider a sentence: ab



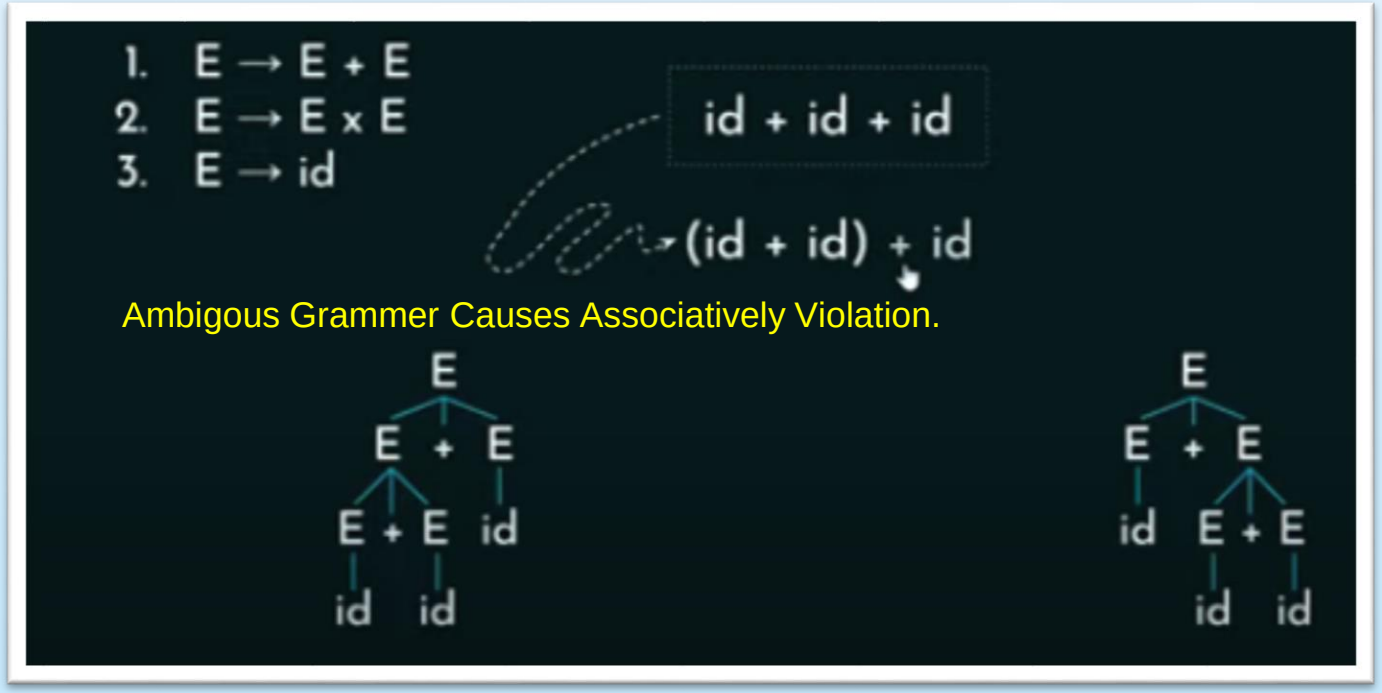
Ambiguity

- Ambiguity Problems:
  - Precedence Property Violation
  - Associatively Property Violation



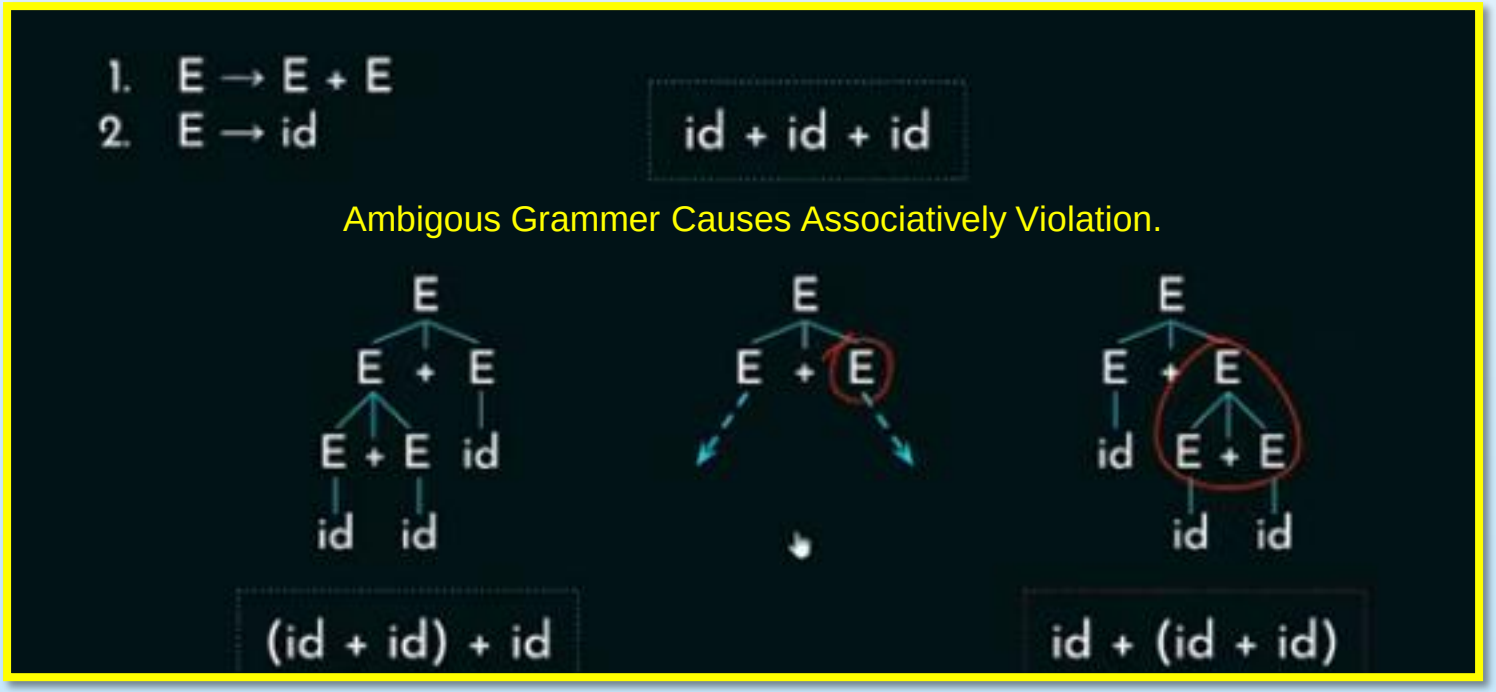
Ambiguity

- Ambiguity Problems:
  - Precedence Property Violation
  - Associatively Property Violation



Ambiguity

- Ambiguity Problems:
  - Precedence Property Violation
  - Associatively Property Violation

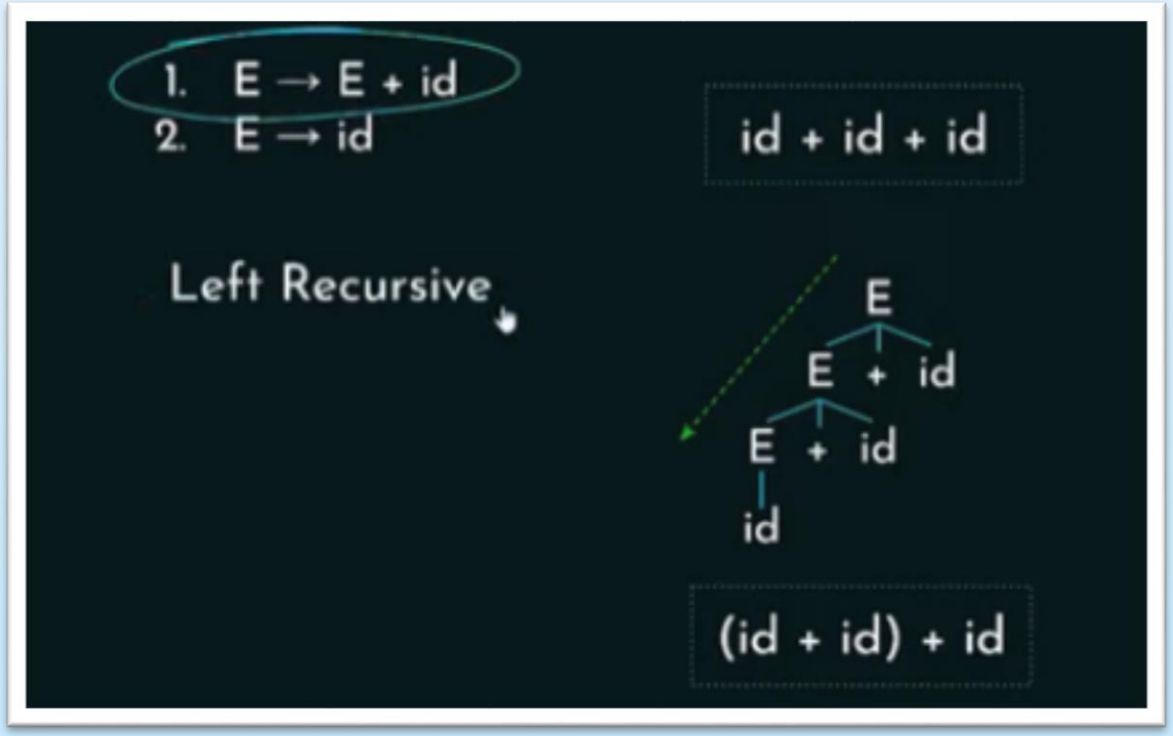


Ambiguity

- Ambiguity Problems:
  - Precedence Property Violation
  - Associatively Property Violation (Solution)
    - Left Recursion = NT on the left of P = Left association
    - Right Recursion = NT on the right of P = Right association

Ambiguity

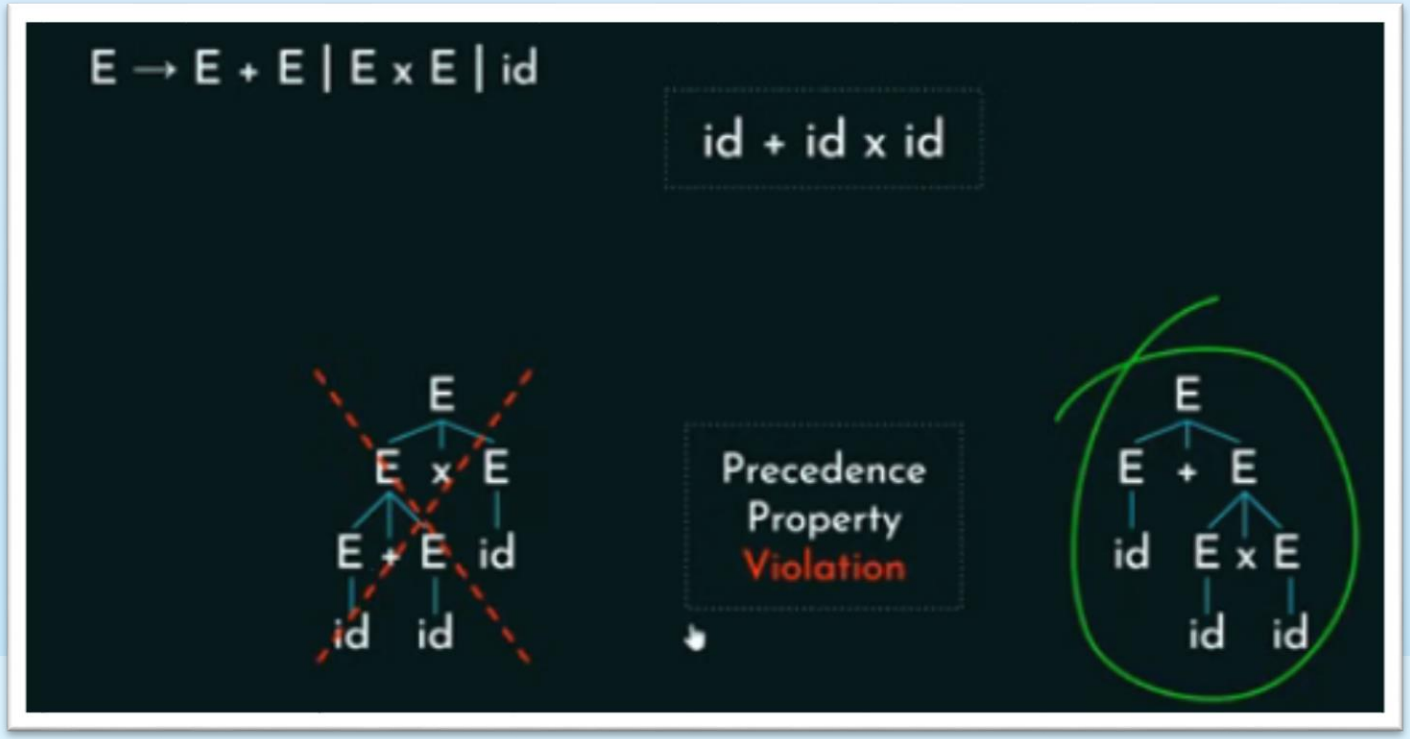
- Ambiguity Problems:
  - Precedence Property Violation
  - Associatively Property Violation



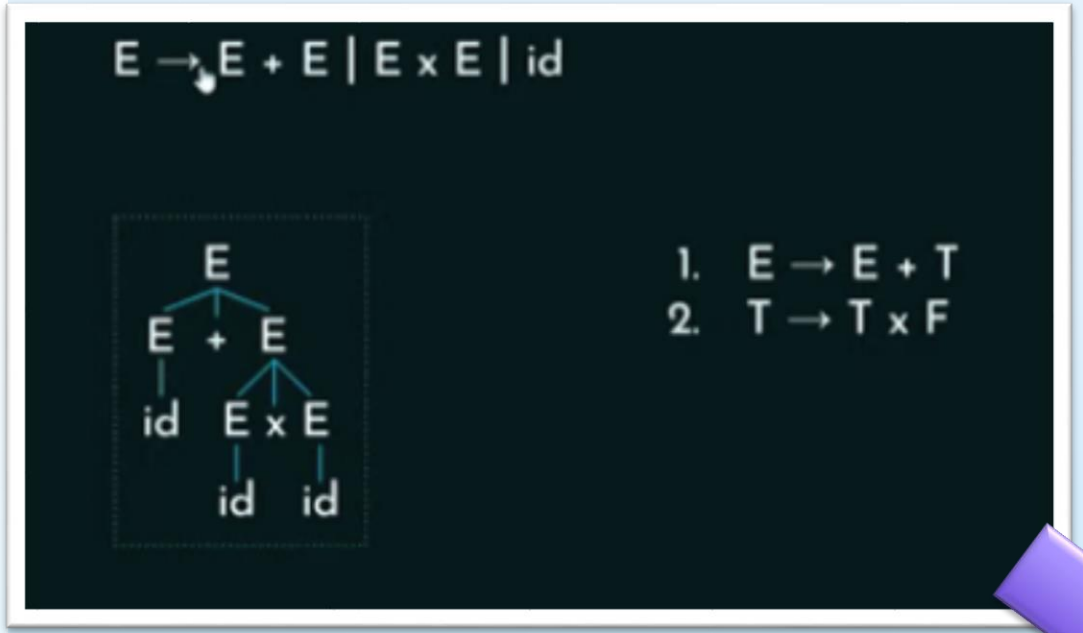
Ambiguity

- Ambiguity Problems:
  - Precedence Property Violation (Solution)
  - Associatively Property Violation

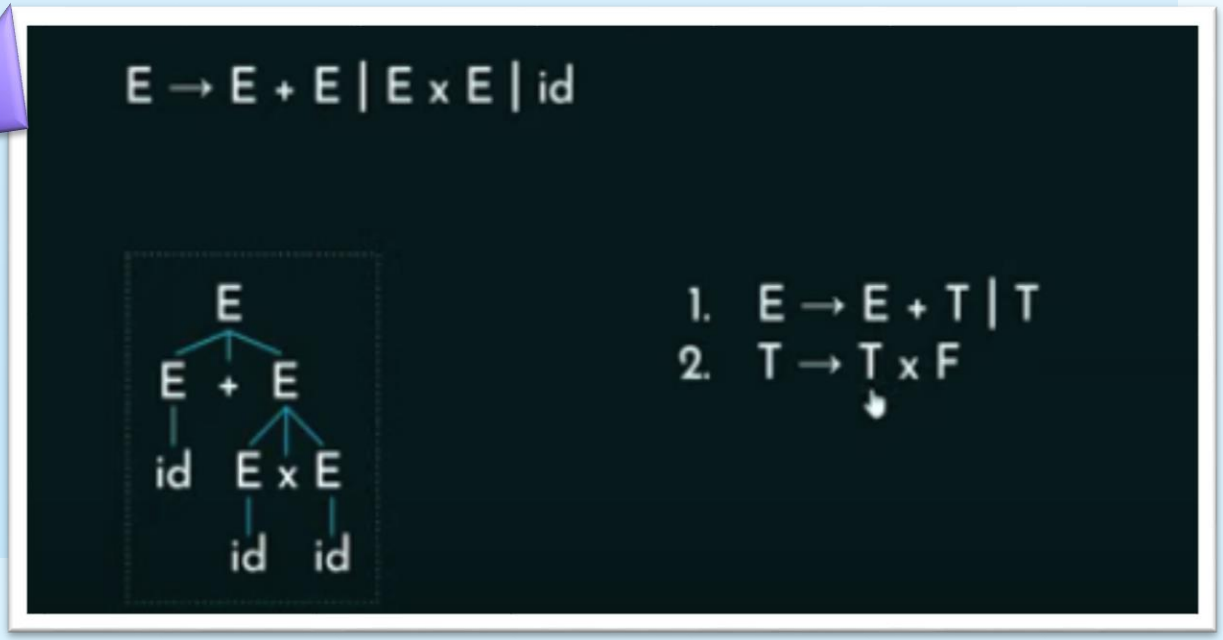
- Ambiguity Problems:
  - Precedence Property Violation
    - Tree is traversed from bottom to top and from left to right. The correct results are produced when the operator of higher precedence is in the lower level.
  - Associatively Property Violation



Ambiguity



Precedence Property



Ambiguity

1. $E \rightarrow E + T \mid T$
2. $T \rightarrow T \times F \mid F$
3. $F \rightarrow id$

Precedence Property resolved by defining levels

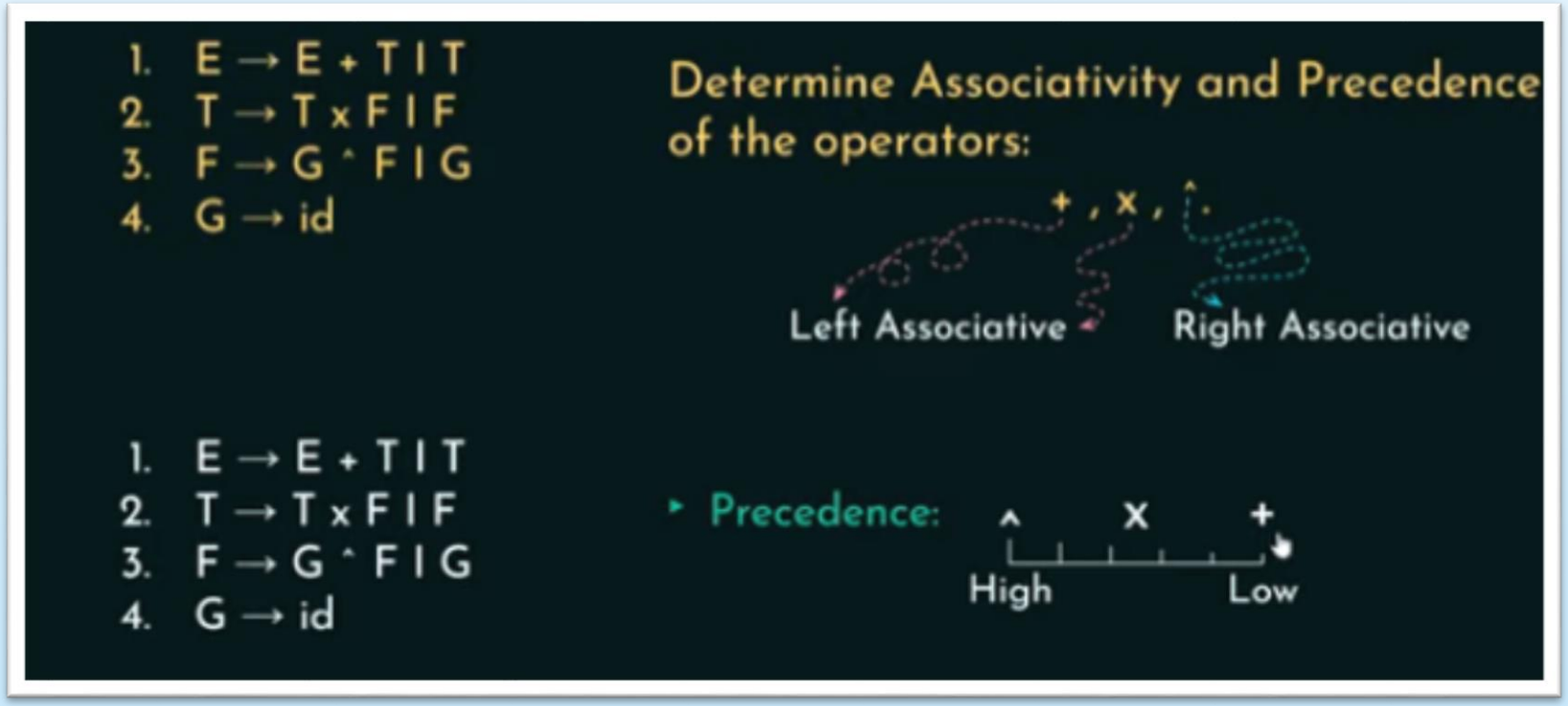
Example


1. E → E + T I T
2. T → T x F I F
3. F → G ^ F I G
4. G → id
Determine Associativity and Precedence of the operators:
+ , x , ^.
Left Associative
1. E → E + T I T

Example


1. E → E + T | T
2. T → T x F | F
3. F → G ^ F | G
4. G → id
Determine Associativity and Precedence of the operators:
+ , x , ^
Left Associative
Right Associative
1. E → E + T | T
2. T → T x F | F
3. F → G ^ F | G

Example


1. E → E + T | T
2. T → T x F | F
3. F → G ^ F | G
4. G → id
Determine Associativity and Precedence of the operators:
+ , x , ^
Left Associative
Right Associative
1. E → E + T | T
2. T → T x F | F
3. F → G ^ F | G
4. G → id
Precedence:
^ x +
High
Low

Example

Consider the following ambiguous CFG on boolean expressions:

bExp → bExp AND bExp | bExp OR bExp | NOT bExp | True | False

The precedence of the boolean operators are NOT, AND, OR (high to low). AND, OR have left to right associativity.

1. bExp → bExp OR bExp
2. bExp → bExp AND bExp
3. bExp → NOT bExp
4. bExp → True
5. bExp → False

Example

Consider the following ambiguous CFG on boolean expressions:

bExp → bExp AND bExp | bExp OR bExp | NOT bExp | True | False

The precedence of the boolean operators are NOT, AND, OR (high to low). AND, OR have left to right associativity.

1. bExp → bExp OR bExp1
2. bExp1 → bExp1 AND bExp2
3. bExp2 → NOT bExp2
4. bExp2 → True
5. bExp2 → False

Example

Consider the following ambiguous CFG on boolean expressions:

bExp → bExp AND bExp | bExp OR bExp | NOT bExp | True | False

The precedence of the boolean operators are NOT, AND, OR (high to low). AND, OR have left to right associativity.

1. bExp → bExp OR bExp1 | bExp1
2. bExp1 → bExp1 AND bExp2 | bExp2
3. bExp2 → NOT bExp2
4. bExp2 → True
5. bExp2 → False

Example

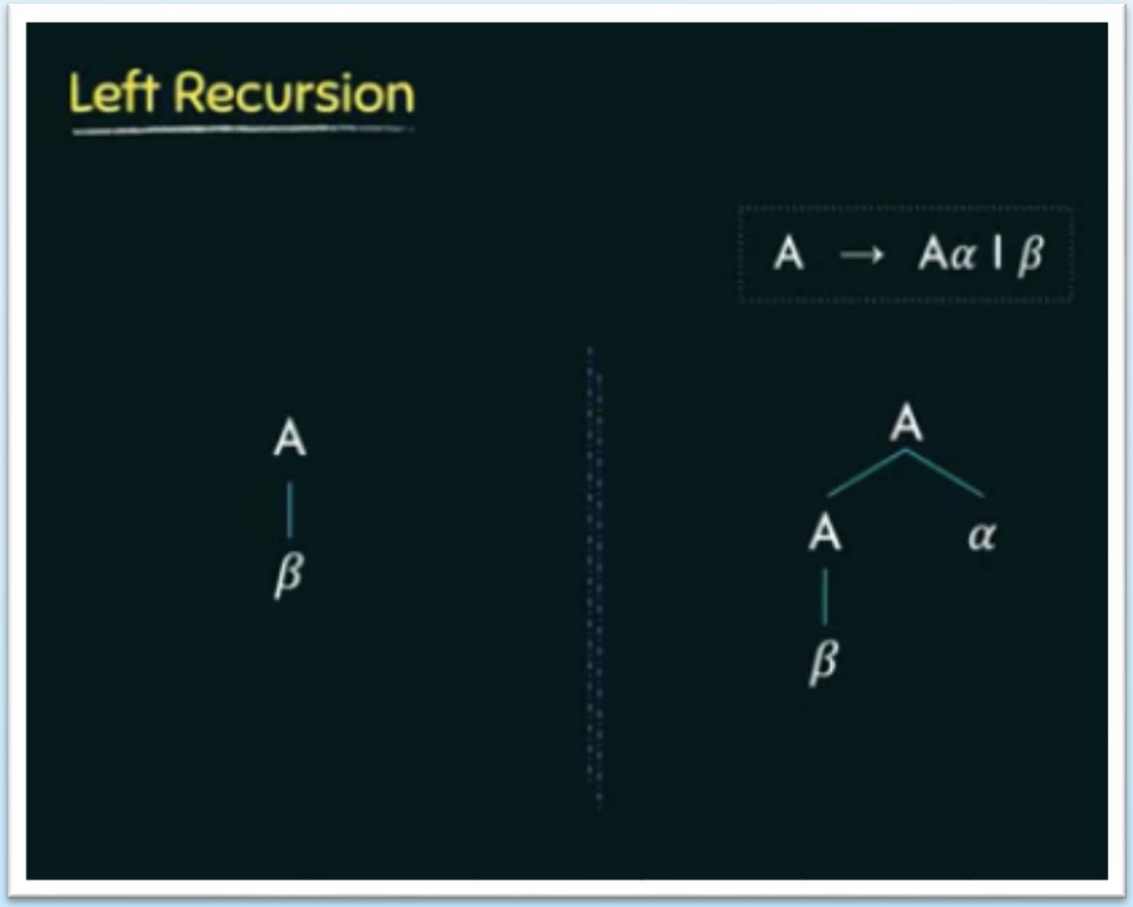
Consider the following ambiguous CFG on boolean expressions:

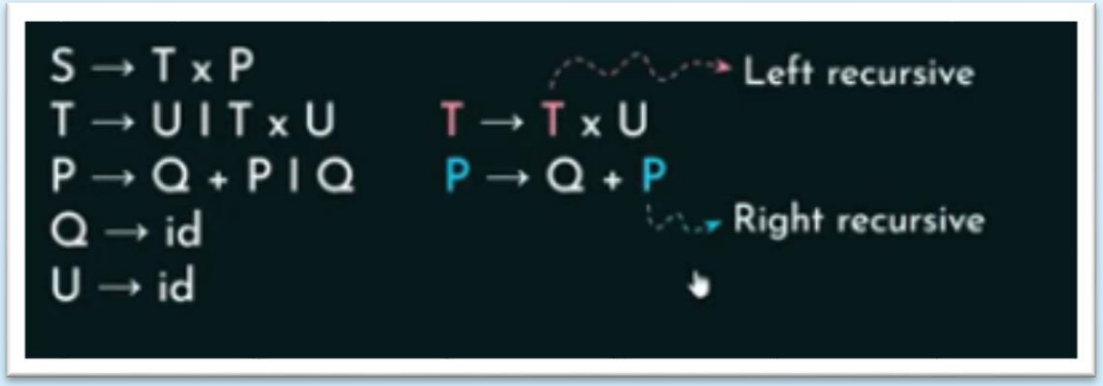
bExp → bExp AND bExp | bExp OR bExp | NOT bExp | True | False

The precedence of the boolean operators are NOT, AND, OR (high to low). AND, OR have left to right associativity.

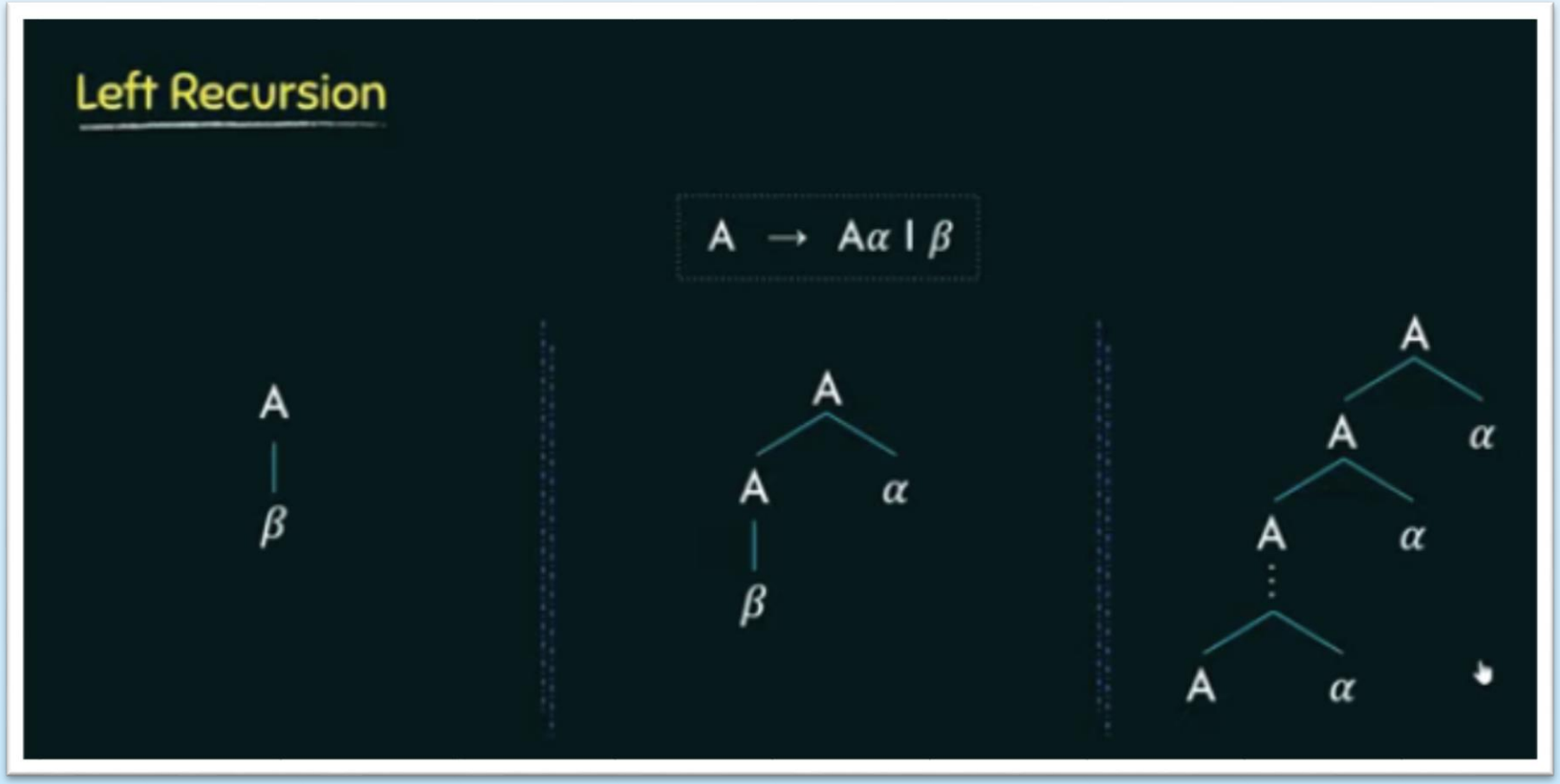
1. bExp → bExp OR bExp1 | bExp1
2. bExp1 → bExp1 AND bExp2 | bExp2
3. bExp2 → NOT bExp2 | True | False

Left and Right Recursions

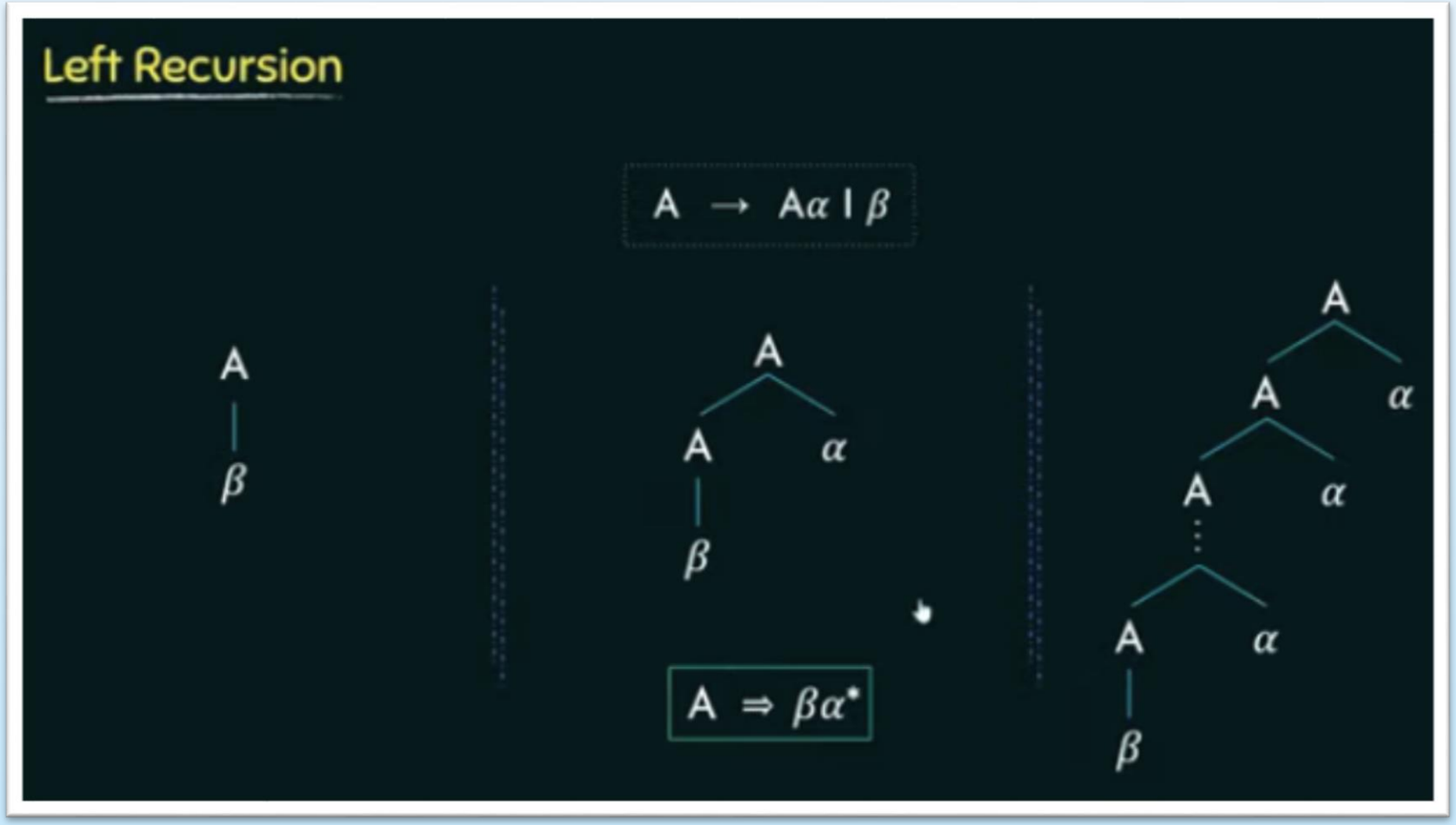




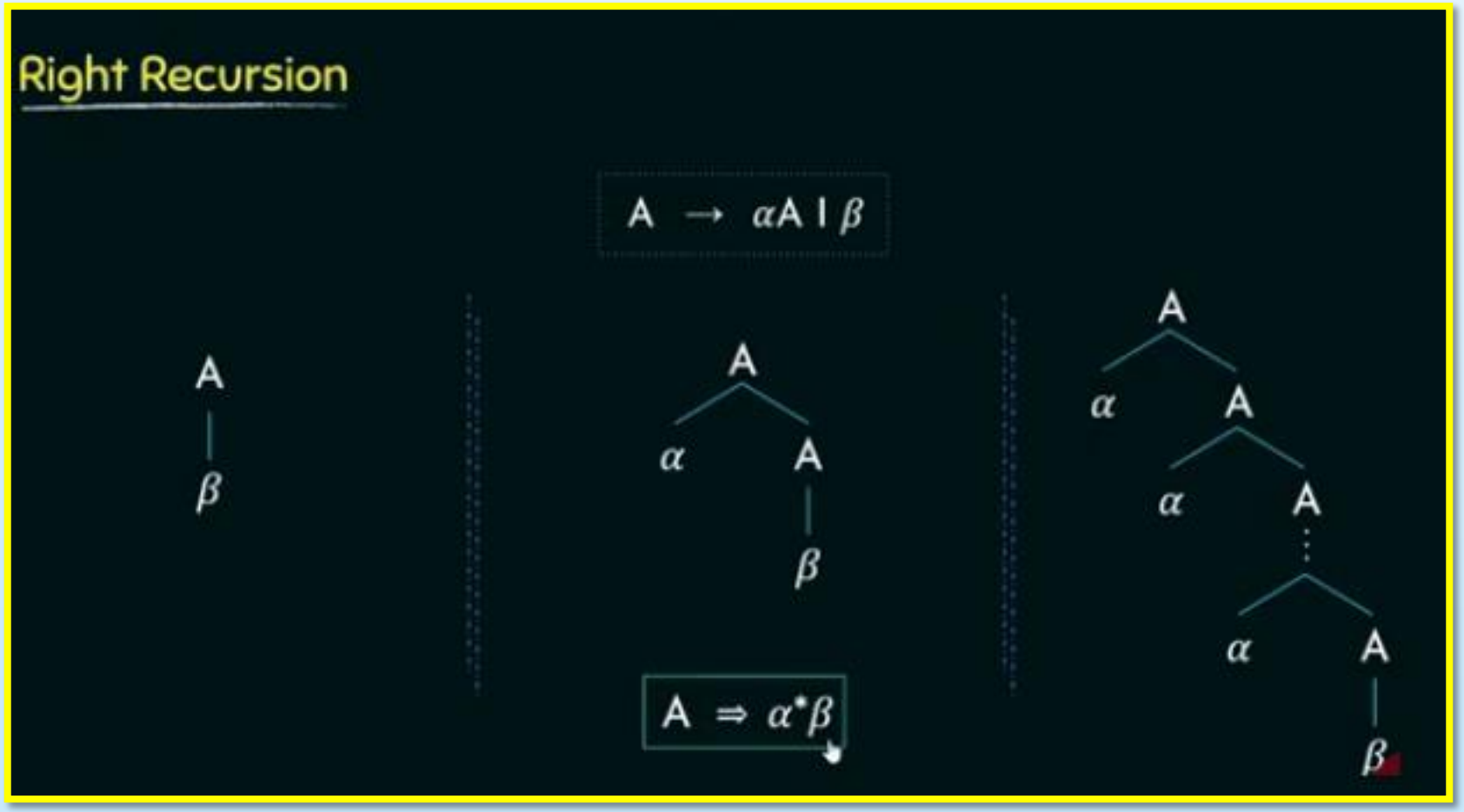
Left and Right Recursions



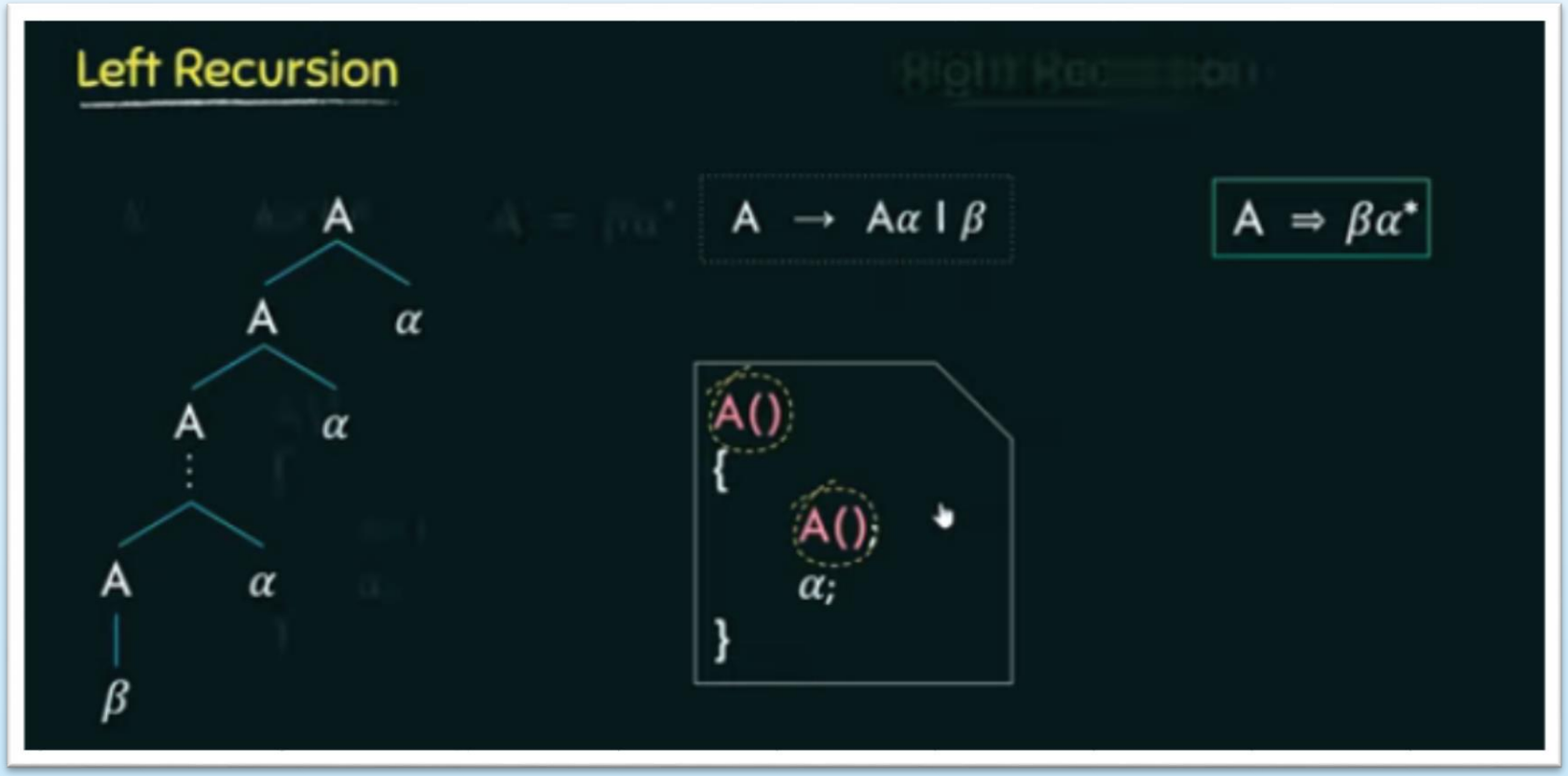
Left and Right Recursions



Left and Right Recursions

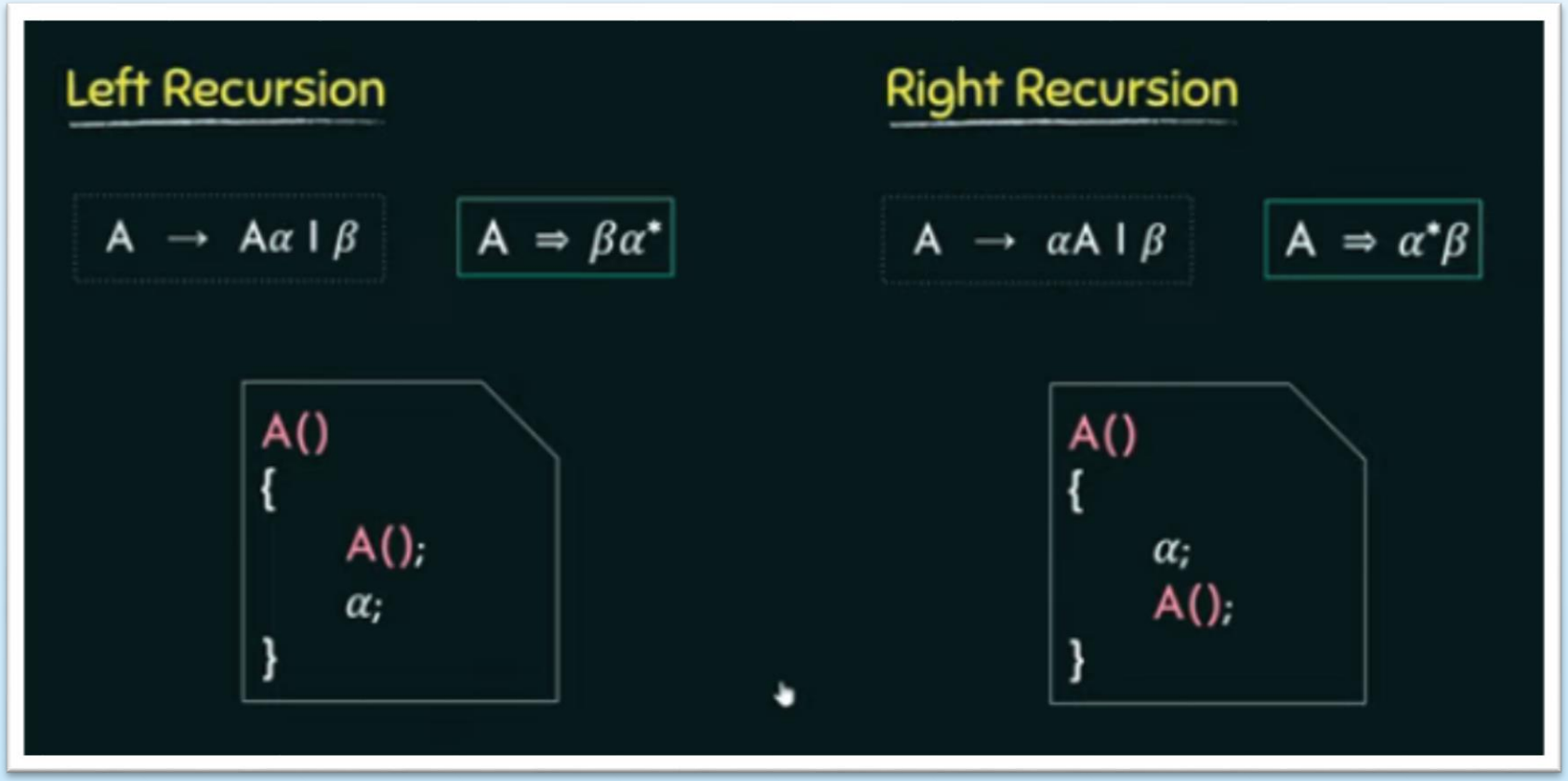


Left and Right Recursions

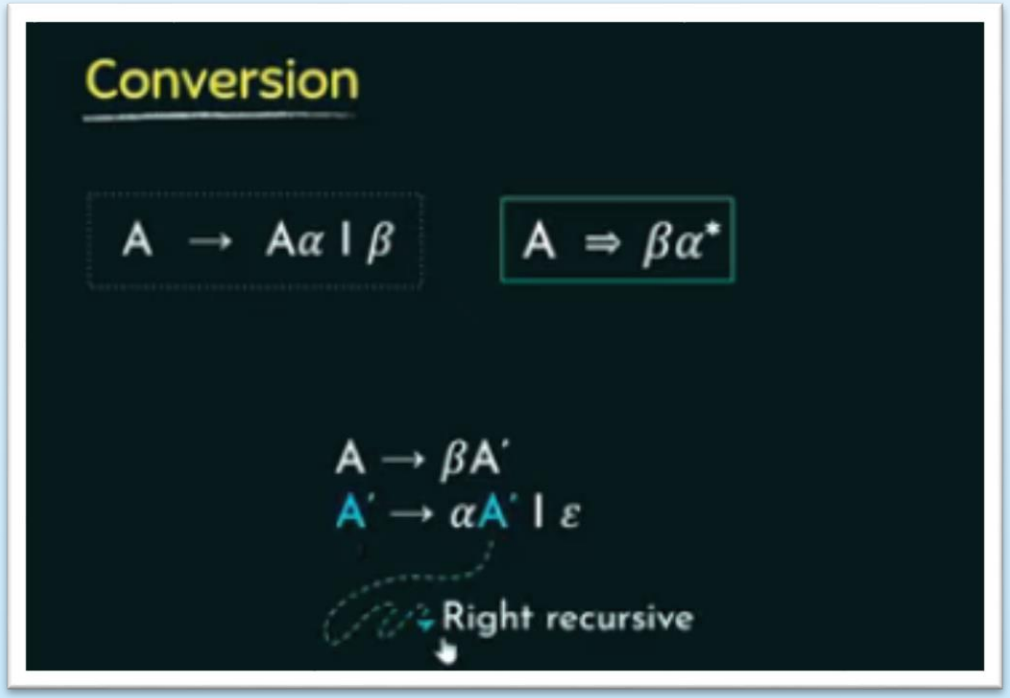


Problem with Left recursion is the infinite loop and confuses the top-down parser

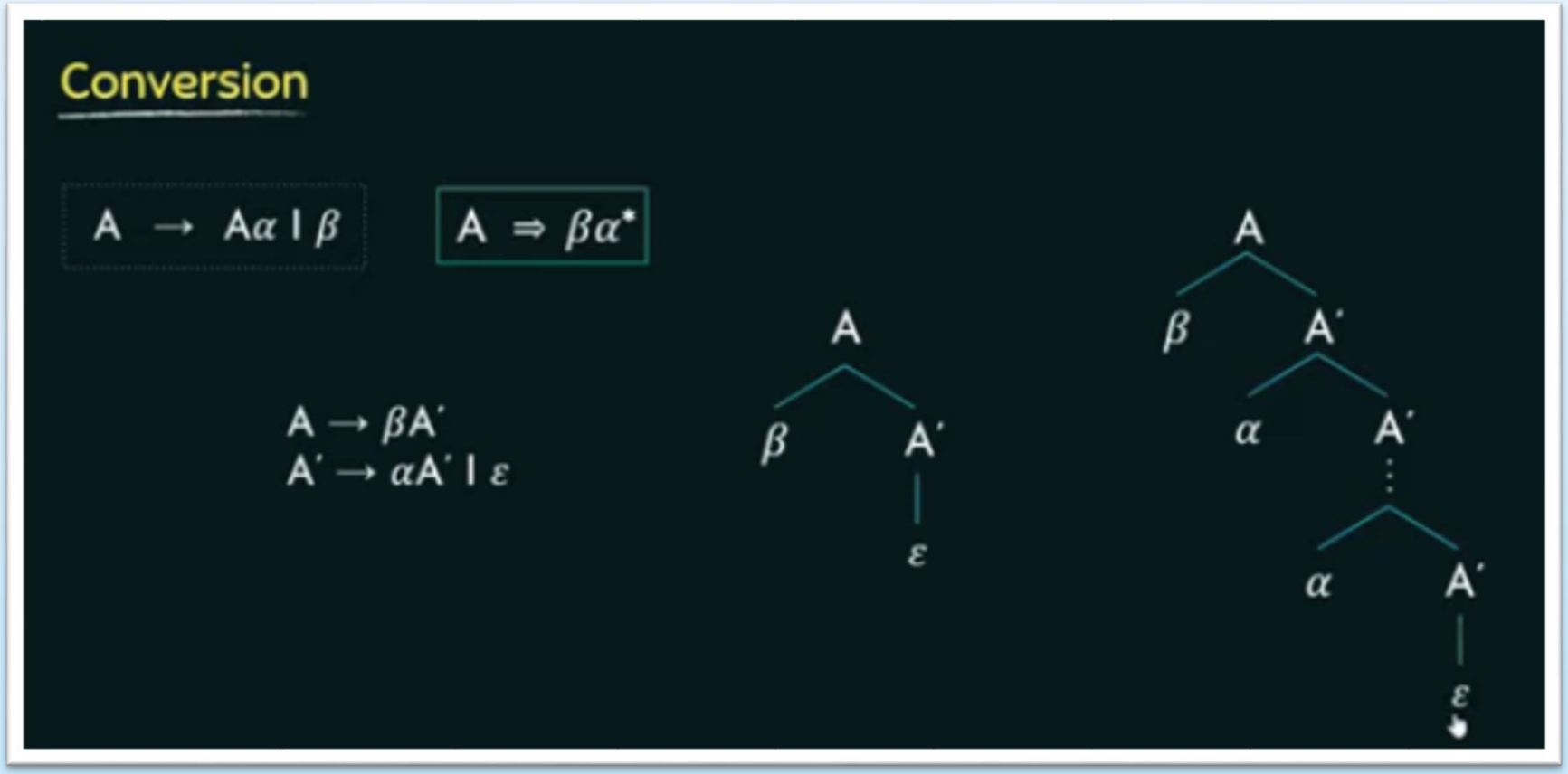
Left and Right Recursions



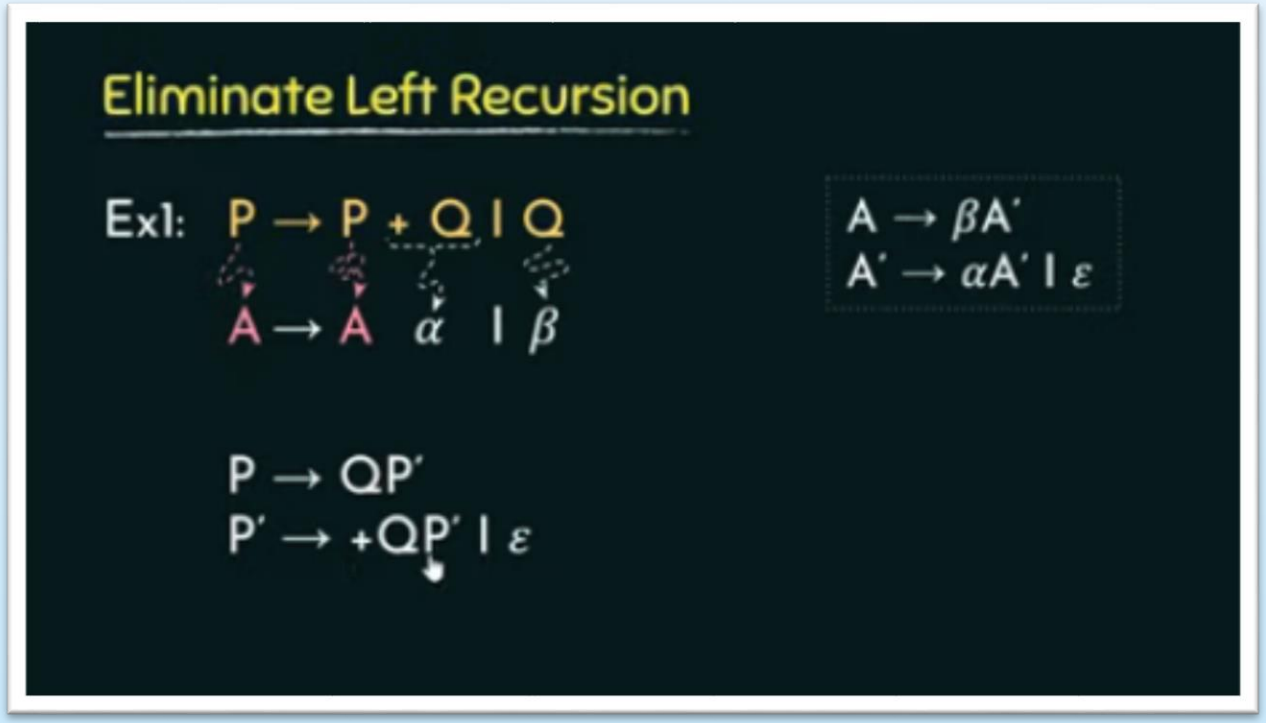
Left Recursion (solution: Right Recursion)



Left Recursion (solution: Right Recursion)



Left Recursion (solution: Right Recursion)



Parse Tree Construction Approaches

- Two distinct and opposite approaches for constructing the tree:
  - Top-Down Parsers begin with the root and grow the tree toward the leaves. At each step, a top-down parser selects a node for some nonterminal on the lower fringe of the partially built tree and extends it with a subtree that represents the right-hand side of a production that rewrites the nonterminal.
  - Bottom-Up Parsers begin with the leaves and grow the tree toward the root. At each step, a bottom-up parser finds a substring of the partially built parse tree's upper fringe that matches the right-hand side of some production; it then builds a node for the rule's left-hand side and connects it into the tree.

Thanks!